

ARC-AGI v2: Explainable Reasoning Traces for GPT5-Nano; The Execution Bottleneck

Isabella Shapland¹ and Jackson Stokes²

¹University of California, Santa Cruz

²Trainloop (YCW25)

ABSTRACT

This work introduces an explainable, step-by-step "instruction booklet" framework for solving ARC-AGI v2 puzzles with GPT5-nano. These "booklets" are comprised of textual multi-step instructions, each step paired with a matching intermediate visual. They are modeled after instruction pamphlets to build furniture, as a tool for analyzing the strengths and bottlenecks of model performance on ARC puzzles. The system first performs a comprehensive analysis of each puzzle's training pairs to generate general steps that take all examples from input to output state. This analysis uses the multi-modal functionality of GPT5-nano, which views the grid state as an image as well as textually. The visuals are created via a crop-transform-uncrop pipeline. On a random set of 33 puzzles labeled by an external human annotator, reasoning was rated fully correct in 16 cases, but execution produced the correct test output in only 1 of those, yielding a 6.2% success rate when reasoning is correct and 3.0% overall success rate. In 16/33 cases, the reasoning was correct but there were issues representing the intermediate images, signifying a gap between the model's understanding and execution of the puzzle. Conversely, no puzzles exhibited incorrect reasoning with correct execution, suggesting that successful outputs almost never arise from "lucky" execution over bad reasoning. Failure-mode analysis shows that when reasoning is correct but execution fails, errors are dominated by visual representation, which may be a product of the visual representation pipeline itself. The main shortcomings were persistent bounding-box and cropping misalignments, continued execution of crop-transform steps even after the puzzle was effectively solved, and the lack of richer visual tools for precise measurement, computation, and annotation during execution. Overwhelmingly, the main failure modes are execution related. These results support a reasoning-vs-execution bottleneck hypothesis for ARC-style tasks and motivate future work on richer visual tools and structured visual editors that can faithfully implement already-correct algorithmic plans.

Keywords: ARC-AGI v2, Visual Reasoning, Step-By-Step Reasoning Traces

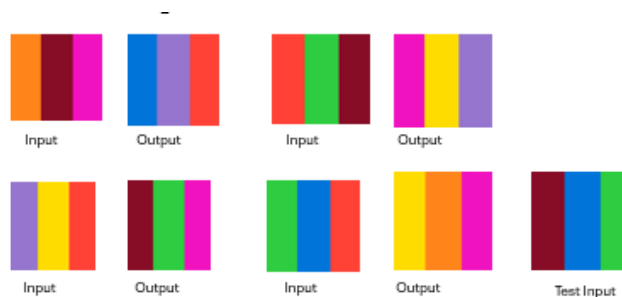


Figure 1. Can you solve puzzle 1?

INTRODUCTION

The ARC-AGI-2 benchmark is a challenging test of reasoning that highlights a stark gap between human and AI performance: top AI models rarely exceed 20-30%, while humans consistently achieve scores in the 80-100% range. This benchmark consists of novel visual puzzles, each presented as a handful of input-output training examples (typically 2-5), followed by test cases where only the input is given and

models must predict the output. Unlike most benchmarks, ARC-AGI-2 deliberately restricts the validity of previously learned solutions, demanding genuine generalization, flexible reasoning, and adaptation from minimal examples. Current models struggle precisely because their reasoning is bound by patterns seen during training, whereas humans intuitively abstract and apply new strategies. Reasoning models also struggle when they are expected to get straight from input to output, which occurs in many state of the art solutions.

0.0.1 Structure

This project aims to design a "traceable reasoning" framework for solving such puzzles, inspired by the experience of assembling furniture. When building furniture, most people rely on detailed furniture instruction booklets: step-by-step guides that combined images and text to make each connection clear and unambiguous. The process is almost automatic, thanks to the traceable and modular nature of the instructions. Thus came the idea of structuring a model's reasoning process after furniture "booklets." The key strengths of this implementation are traceability, broken down tasks, and visual reasoning. Because this implementation allows traceability, we can look at each step to see where a model's reasoning fails, and what kind of 'step descriptions' produce the best output. The model also breaks down the problem into many steps, allowing operations to be modular. This improves the accuracy of a single step, since GPT5 can easily perform small operations like "fill in the cell surrounded in a green border." This implementation also uses visual analysis alongside textual analysis on puzzle examples, which allows for better understanding of the puzzle and more generalized step descriptions.

0.0.2 Failure Modes and Implications

This traceable reasoning structure produced concrete failure-modes and highlighted strengths and weaknesses of GPT5-nano on the ARC dataset. In this study correct outputs never arose from incorrect reasoning, so success nearly always reflected genuinely correct algorithmic understanding. Further, when independent human annotators rate the reasoning as fully correct, execution still fails in 93.8% of those puzzles, usually due to visual representation errors like bounding-box misalignment or step count mistakes. This suggests that models have the capability to understand and generalize step-by-step solutions to ARC puzzles, but struggle to execute their reasoning visually. According to the official ARC website, GPT5-nano's reported raw accuracy on ARC-AGI-2 is 0.9% while our reasoning correctness score stood at around 45.5%, exemplifying the strength of visual and textual comprehensive analysis. This analysis, along with the strict IF-THEN template of each step, allowed the model to fully encapsulate the reasoning required within the puzzle into 5-10 generalized steps. GPT-5-nano was chosen to demonstrate that even a comparatively weak ARC solver can still grasp the underlying puzzle logic when given rich image-and-text analyses. The problem may be visual execution.

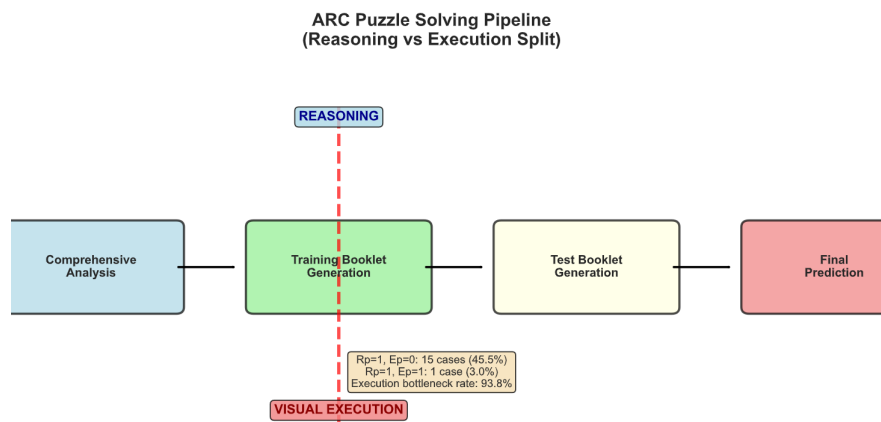


Figure 2. Pipeline with Bottleneck

STATE OF THE ART APPROACHES

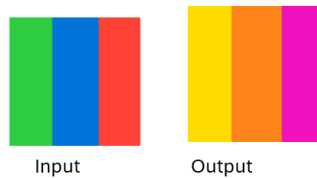
The most recent approaches to solving the Abstract Reasoning Corpus (ARC) involve natural language, evolving brute force attempts, and visualization. Jeremy Berman's highest scoring solution (29%) involves using sub agents to execute natural language instructions. The author proposes a solution that then iteratively refines the instructions via two revision modes: individual and pooled. Individual revisions refine a single instruction using cell diffs between predicted and true outputs, while pooled revisions synthesize a new instruction from several high-performing ones. I was most inspired by Berman's use of natural language to define the input-output transitions, since code can be brittle and neurosymbolic reasoning can be difficult to define. The main difference between this solution and this one is the nature of the instructions. While Berman's solution attempts to define a single instruction that can take you from input to output states over all puzzles in a training example, my solution takes a step-by-step approach. This allows the model to make smaller, more manageable adjustments to the grid. It also creates instructions that fully capture the intricacies of the puzzle's "trick".

Secondly, I was deeply inspired by chain-of-thought reasoning structure. Feng et al. introduce *Self-motivated Learning*, where a model automatically generates multiple rationales for existing reasoning datasets and then uses answer correctness to induce a preference ranking over these rationales. A reward model is trained on this implicit preference signal and used within a PPO framework to fine-tune a 7B LLaMA 2 model, leading to significant gains on arithmetic and commonsense benchmarks. In contrast, the approach in this paper targets ARC-AGI v2 with explicitly visual, step-wise "booklet" instructions over grids, aiming for traceable and human-interpretable reasoning rather than purely textual rationales, but was inspired by this papers reasoning algorithms.

The third solution from which I drew inspiration was Zoe Carver's ARC-Saturn repository. The accuracy achieved in this attempt is 22%, a far cry from full accuracy but much better than 15.9% that Claude achieves on its own. This solution involves using a multi-modal model to view the image representations of ARC puzzles, and tool-calling for the model to create it's own visualizations. One of the main inhibitors of LLM's performance on ARC puzzles is their inability to "see" the grid the way humans do. The format of a puzzle in the ARC dataset is a matrix of numbers, each corresponding to a color.

$$\text{input : } \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} \rightarrow \text{out put : } \begin{bmatrix} 4 & 5 & 6 \\ 4 & 5 & 6 \\ 4 & 5 & 6 \end{bmatrix}$$

Is represented visually as:



Carver's solution allows the model to see the puzzles visually alongside these grid representations. The model can then perform grid transformations with the visual information that humans already have. Carver remarks, "GPT-5 actively uses the visualization tool to test hypotheses, showing exploratory behavior, and frequently assigns meaningful semantic labels to patterns like "onion layers," "rooms and corridors," or "anchor points," suggesting it's building abstract representations rather than just processing pixels." Visual representation creates a more abstract and generalizable understanding of the puzzle. This solution uses the same visualization tool-calls and multi-modal application. The main difference of this solution is that the generated visualizations have a clear step-by-step structure, whereas in Carver's solution the model decides when to produce visualizations. Carver's model visually predicts the output to each training input individually, then maintains conversation history while predicting the output of the test input. This way it can "see" what solutions worked for the training examples. The model is still expected to get from input to output in one step. Carver's documentation specifies that working solutions "emerge from breaking problems into sub-components." This is the main philosophy of this solution.

METHODS AND MATERIALS

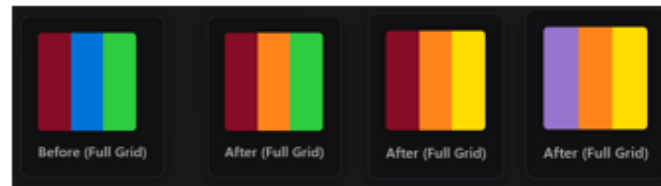


Figure 3. Predicted (correct) Test booklet for Puzzle 1.

The method I use in the generation of step-by-step "booklets" involves 6 stages.

- A comprehensive analysis of all training inputs and outputs
- Textual step generation
- Visual step generation
 - Matches regions of the puzzle to change in this step
 - Performs transformation for each region guided by training output
 - Moves to next step
- Analysis of training booklets
- Test textual step generation
- Test visual step generation
- Final output

0.0.3 Comprehensive Analysis + Generated Steps

The first stage requires GPT5-nano to perform a thorough comparative analysis over the entire set of training examples. The model systematically compares all training inputs, all training outputs, each input-output pair in isolation, and then each pair against every other pair, examining every grid and image representation for shared structure and key differences. Using this analysis, general steps that can take all inputs to their output state are generated.

The first step always mentions a grid size to generate. There is also a step that mentions which objects are unchanged. The following steps are then formatted as an IF (condition) THEN (perform x on y). The "x" in this format refers to a transition, whether that be moving, changing the color of, creating or some other change on a part of the grid. The "y" in this format refers to some object, section, section of object, or line that experiences a change from input-output state. This creates steps that modify the grid portion by portion based on some condition.

The model will skip steps when no eligible region is found that matches the condition. Each step is also tagged with either OBSERVE or TRANSFORM, depending if the step calls specific objects into focus for reference, or is actually performing modifications on the grid. Each step matches all regions/objects that fit the given condition, and changes them one by one.

The step generation stage generates "booklets" one training pair at a time. Depending on the input-output size ratio, the model will either start with the input grid size or the output grid size. It will start with the input grid if the output grid size is of the same dimensions, and start with the empty output grid size if the input and output differ in dimensions.

On OBSERVE steps, the model draws bounding boxes over the input objects mentioned in the step. This way the model can reference the input images in the training booklet without changing them, even when working on the output-sized grid.

On TRANSFORM steps, the model will crop to the first object that matches the IF (condition) and perform the specified transition. This step is guided by the output grid cropped to the same area. (See Figure 5 for example, where substep 3.1.2 is guided by "target" output.) It will then "uncrop" to the main grid, leaving the object transformed to its output state while all other objects remain the same. It will then go on to the next region that is transformed in the final output. The steps continue for each object that needs to be changed in a step. Once all objects that satisfy the given condition are transformed, the model moves onto the next step.

0.0.4 Failure Modes

The biggest issues stem from this section of the pipeline. Observe/Transform steps were added to better fit the steps where specific objects were measured or called into reference. This was in effort to add to the rich toolbox required to faithfully represent some of the generated steps. The object recognition can be faulty, causing misaligned cropping, steps that missed or repeated objects, or vague bounding boxes that fit over the entire puzzle. This was the main category of failures. The other labeled failure-modes include

Category A: accuracy errors (A1 complete test failure, A2 partial test failure, A3 training failure),

Category B: Step count issues (B1 too many steps, B2 too few steps) and

Category C: Visual Fidelity issues (C1 text reasoning mentions visual grid inaccurately, C2 bounding box/cropping issues, C3 is any visual that does not look like the generated step.)

Most failures with that are labeled as correct reasoning but failed execution have errors A1, A3, B1, C2 and C3. This means the mistakes here are from too many objects per step, bbox misalignment, cropping mistakes, training failures, and visuals that don't match their text descriptions. See results for failure mode statistics. With the context of the current implementation structure and the common failures, we can proceed to examples. The steps generated for the Figure 1 example are below.

Example 1 Generated Steps:

- Step 1: IF the input is a 3x3 block THEN keep the grid size identical (do not change the number of rows or columns) and preserve each cell's position.
- Step 2: IF a cell's color is 1 THEN change color to 5.
- Step 3: IF a cell's color is 5 THEN change color to 1.
- Step 4: IF a cell's color is 2 THEN change color to 6.
- Step 5: IF a cell's color is 6 THEN change color to 2.
- Step 6: IF a cell's color is 3 THEN change color to 4.
- Step 7: IF a cell's color is 4 THEN change color to 3.
- Step 8: IF a cell's color is 8 THEN change color to 9.
- Step 9: IF a cell's color is 9 THEN change color to 8.
- Step 10: IF a cell's color is not one of 1,5,2,6,3,4,8,9 THEN leave that cell's color unchanged.

The steps generated in this component consistently show deep understanding of the puzzle's "trick", regardless if the visual generation steps were able to predict the test output correctly.

RESULTS

This is the comprehensive, start to end solving pipeline for puzzle 2 starting with the initial inputs/outputs.

Example 2 Generated Steps:

- Step 1: IF a connected region forms a hollow square border (i.e., pixels of color is 2 arranged as a 1-pixel-thick square loop with an enclosed rectangular area) THEN treat that region as a frame object to be processed independently.

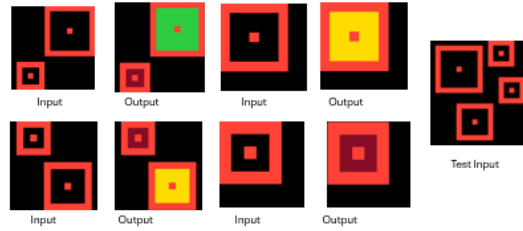


Figure 4. Puzzle 2

Step 2: IF a frame object (color is 2) is identified THEN compute its outer bounding-box size $S = (\max_row - \min_row + 1)$ (or equivalently outer width in pixels) to classify its ring size.

Step 3: IF the frame object's outer bounding-box size $S = 5$ THEN fill the frame's enclosed area by changing every enclosed cell whose color is 0 to color 8 (IF enclosed cell color is 0 THEN change color to 8).

Step 4: IF the frame object's outer bounding-box size $S = 7$ THEN fill the frame's enclosed area by changing every enclosed cell whose color is 0 to color 4 (IF enclosed cell color is 0 THEN change color to 4).

Step 5: IF the frame object's outer bounding-box size $S = 9$ THEN fill the frame's enclosed area by changing every enclosed cell whose color is 0 to color 3 (IF enclosed cell color is 0 THEN change color to 3).

Step 6: IF a cell is part of a frame (color is 2) OR a cell inside a frame is nonzero (color is not 0) THEN do not change that cell's color (preserve all color-2 border pixels and any nonzero interior pixels).

Step 7: IF multiple disconnected hollow square frame objects (color is 2) exist in the grid THEN apply Steps 2–6 independently to each frame; do not change any 0 cells that are not enclosed by a color-2 frame.

0.1 Visualized Step generation

Using these instructions, the next module will create "booklets" for each training example. Visualizations are created when objects are focused on in a step (OBSERVE) or changed in a step (TRANSFORM). The first two steps are observe steps, mentioning what constitutes a changeable object and what measurements to compute. These steps mistakenly skipped the visual generation since they were falsely labeled transform steps (with no real transformation) instead of observe steps, but the step description was noted by the training model when generating the rest of the steps, and the test model when determining the final output.

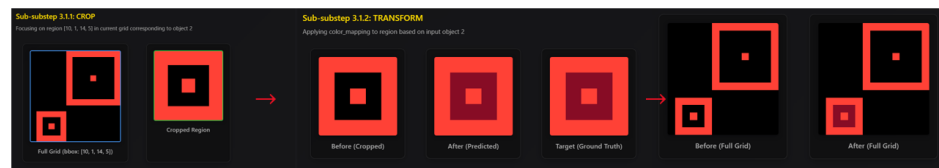


Figure 5. Step 3, Crop → Transform → Uncrop

Step 3: IF the frame object's outer bounding-box size $S = 5$ THEN fill the frame's enclosed area by changing every enclosed cell whose color is 0 to color 8 (IF enclosed cell color is 0 THEN change color to 8). The model then picks the qualifying object (bounding box size $S = 5$) and performs the transformation. No qualifying objects were found for step 4, so the step was skipped.

Step 5: IF the frame object's outer bounding-box size $S = 9$ THEN fill the frame's enclosed area by changing every enclosed cell whose color is 0 to color 3 (IF enclosed cell color is 0 THEN change color to 3).

This is the completed puzzle. No static objects were mistakenly modified thanks to Step 6. (IF a

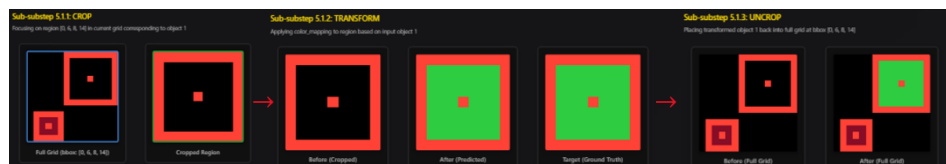


Figure 6. Step 5, Crop → Transform → Uncrop

cell is part of a frame (color is 2) OR a cell inside a frame is nonzero (color is not 0) THEN do not change that cell's color (preserve all color-2 border pixels and any nonzero interior pixels.) Since there can be up to five slightly different training examples for each puzzle, some steps promote generalization like Step 7. *IF multiple disconnected hollow square frame objects (color is 2) exist in the grid THEN apply Steps 2–6 independently to each frame; do not change any 0 cells that are not enclosed by a color-2 frame.* The test model utilizes steps like these when a test output is different than any training example it's seen.

0.2 Completed Training Booklets

The visualizations part of the training booklets that the model sees are presented below. Each training booklet uses the same general steps and each visual is accompanied by the step description.

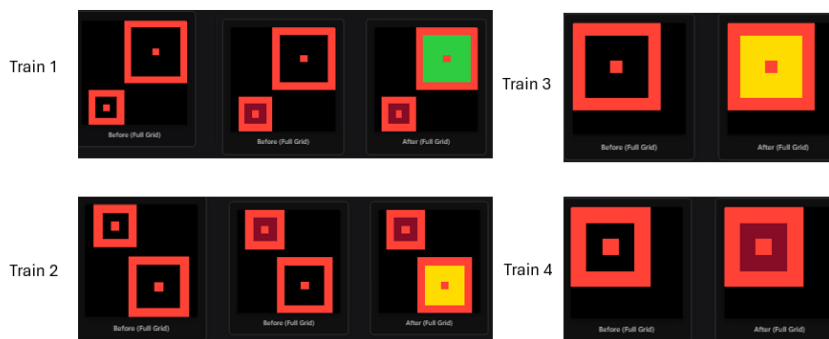


Figure 7. Training booklets 1-4 For Puzzle 2



Figure 8. Training Booklets 1-4 for Puzzle 1

0.3 Test Booklet Generation

Next, given the training booklets and general steps, the test model predicts the test output using the same process as the training booklets. This includes the TRANSFORM and OBSERVE steps, the crop-transform-uncrop substeps for changing regions, and matching objects conditionally per step. This time it is not guided by the test output at all, rather it must rely on the visual and textual information given by it's training. The end result is an explainable reasoning trace for ARC puzzles 0dbd492 and 0d3d703e. The model fills in equivalent objects in the same step, (see red boxes in Figure 8,) or changes object by object when regions experience different transformations (see all other steps.) Having the training booklets is

useful to visually understand the general steps, and the model can use the generated steps to fill in the “gaps” between the training booklets and what a test booklet should look like, (see step 7). It’s also useful to explain the reasoning behaviors of LLMs, what they fail on, and what consistently is successful. (See weaknesses and strengths.)



Figure 9. Test Booklet for Puzzle 2

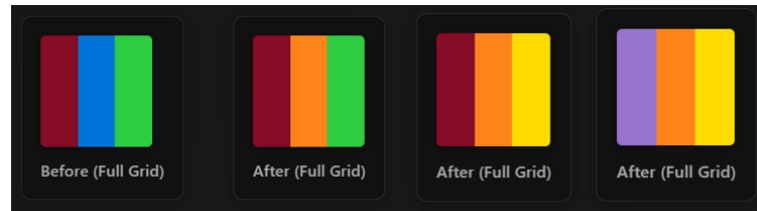


Figure 10. Test booklet for Puzzle 1

DISCUSSION

The results of this experiment are varied. Across 33 annotated puzzles, reasoning was rated fully correct in 16 cases, but execution produced the correct test output in only 1 of those, yielding a 6.2% success rate when reasoning is correct and a 3.0% overall accuracy. While there are a few examples of effective training booklets (100% train, 100% test), the majority of booklets fall into an “ineffective” category where all training booklets reach the correct output but the corresponding test prediction still fails (100% train, 0% test). This section therefore analyzes not only the qualitative strengths and weaknesses of the current implementation, but also how specific structural properties of a booklet correlate with these Rp/Ep outcomes and with the dominant execution failure modes (A1, A3, C2, C3) identified in the failure-mode analysis. Together, these results suggest that the generated steps already capture the puzzle’s algorithmic structure in many cases, but that the visual execution layer is currently the primary bottleneck to achieving high end-to-end accuracy. In the table below it’s clear that reasoning fails much less often than execution does:

Conditional Probabilities: On this randomly selected group of 33 puzzles, execution is the most common factor for failure. Visual execution only worked for 1/33 puzzles.

Probability execution succeeds given correct reasoning: $P(E_p = 1 \mid R_p = 1) = \frac{1}{16} \approx 0.0625$ (6.25%)

Probability execution fails given correct reasoning: $P(E_p = 0 \mid R_p = 1) = \frac{15}{16} \approx 0.9375$ (93.75%)

Probability execution succeeds given incorrect reasoning: $P(E_p = 1 \mid R_p = 0) = \frac{0}{17} = 0$

Probability execution fails given incorrect reasoning: $P(E_p = 0 \mid R_p = 0) = \frac{17}{17} = 1$

Probability reasoning is correct given failure: $P(R_p = 1 \mid E_p = 0) = \frac{15}{32} \approx 0.46875$ (46.88%)

Probability reasoning is incorrect given failure: $P(R_p = 0 \mid E_p = 0) = \frac{17}{32} \approx 0.53125$ (53.12%)

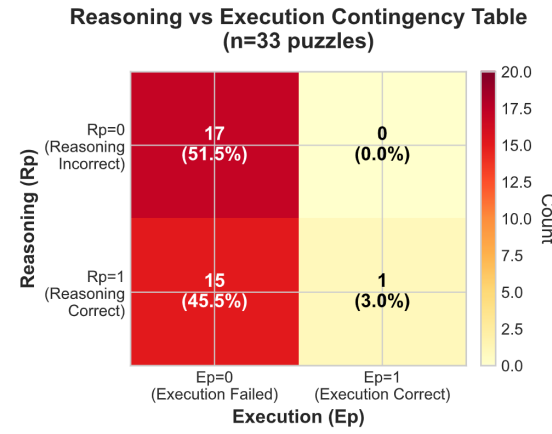


Figure 11. Reasoning vs Execution heatmap

0.4 Weaknesses

There are three main weaknesses to the current implementation.

0.4.1 Difficulty translating NLP instructions into visual steps- C3 issues

The current version has some problems generating steps visually. Since there are multiple steps, and a visual must be generated for each, it leaves a lot of room for error within the visualizations themselves. As a result, sometimes the organization of a complex booklet can repeat steps, select too many objects per step (repeating multiple transitions on one object), or skip steps when it shouldn't. The model can sometimes also just crop to the whole puzzle (seeing the whole puzzle as one object) and paste in the output as a whole. In general, the current generator can often get all training booklets to end with the output state of the given puzzle, but the steps to get there, visually, can sometimes not be descriptive or clear enough. The main difficulty in this experiment was creating visuals that fully captured the given step.

At first, I tried producing visuals without processing by object or cropping, simply handing the model the step description in natural language without having any other kind of structure. It had a lot of difficulty achieving the output state correctly, since the natural language instructions were difficult to implement. Multiple steps also leaves room for error if the model has no reference for how the step should be completed. I had to change both the format of the instructions, and the structure of the visualization generation. The first change I made was to add the "ground truth" or the final output of each training example to the model's context window. Unfortunately adding in the final output as a whole causes the model to skip right to the output. Often it would paste the output entirely for each step, which leads to 100% training accuracy, but a failed test booklet. From here it was clear that the visual generation needed some kind of partial final output of training examples to draw reference from. (Training inputs AND outputs are given to the test taker, so it is completely legal to show the model these training outputs.) Then we came up with the idea of cropping to certain sections of the puzzle and changing the section that was cropped to. This allows the model to make partial changes without changing anything else on the grid by mistake. I then edited the step descriptions to have an IF (condition) THEN (transform) to more easily prompt the region-oriented nature of the new generator. The one drawback of changing the puzzle section by section is that sometimes the puzzle's "trick" or reasoning cannot be represented well by a section-by-section change to the puzzle.

0.4.2 Time

The runtime of this program can be up to 15-20 minutes per puzzle. This is due to the multi-stage nature of the pipeline. Comprehensive analysis can take up to a minute, since the model is viewing both the grid and visual representations of the puzzle. Depending on the number of steps, the training booklet generation can be from 8-12 minutes alone, since the visualization aspect has three components. First is object matching, where an LLM selects the objects that match the step description, then for each object crop, prediction of the object at output state, then uncrop. The final component, generating the test booklet, can be up to 5 minutes. This can be optimized upon revision.

0.4.3 OBSERVE Steps

One last weakness of this version is that OBSERVE steps can be faulty. Sometimes it will put a bounding box over the entire puzzle instead of the specific reference objects to be observed, which is not descriptive. It can also mislabel OBSERVE steps as transform, since the current labeling system uses an LLM to analyze the steps. (It determines if objects are actually changed in this step.) In a future version I will rework the system labeling TRANSFORM vs OBSERVE steps, and this will add a layer of explainability to future training booklets. The end goal is to have OBSERVE steps draw small bounding boxes over each object/region/section/line mentioned in the OBSERVE step, so the training and test models has a reference of what input objects influence the output transitions.

Below is a chart representing the probability of all failure modes, and which failure modes are most common in the case that the reasoning is correct. For reference, the failure modes (which were hand-labeled by an independent annotator) are:

Category A: accuracy errors (A1 complete test failure, A2 partial test failure, A3 training failure),

Category B: Step count issues (B1 too many steps, B2 too few steps and

Category C: Visual Fidelity issues (C1 text reasoning mentions visual grid inaccurately, C2 bounding box/cropping issues, C3 is any visual that does not look like the generated step.)

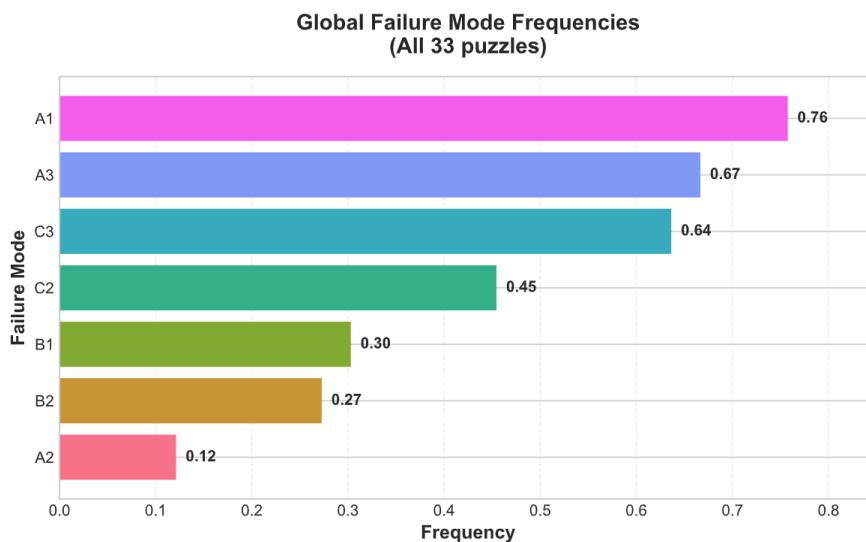


Figure 12. Failure Modes over the whole dataset

Overall, the main failure modes are test and training prediction inaccuracies (A1, A3) as well as C3 visual representation issues (the step visuals do not visually represent the textual description.) After that the most common issues are bounding box/cropping errors (C2), as well as too many or too little steps (B1, B2) as mentioned. Lastly there are some partial test failures, which are interesting because some of the test cases may have been correct/nearly correct execution, but were marked as incorrect since all test cases must reach 100% visual accuracy to be marked correct. In the case that reasoning was correct, the failure modes slightly differ:

There are higher frequency of C3 errors in the case that the reasoning is correct. C3, visual representation errors, might appear more often in these cases due to the complexity of genuinely correct reasoning, exemplifying the need for a rich toolbox for measurement, computation, and object selection/transformation. On a more difficult puzzle, annotator notes, "Got it mostly correct surprisingly. The failed training was due to C2, which was the only example with this issue. Training example 2 suffered from B1." Correctly reasoned cases still have difficulty with the number of objects per step and bounding

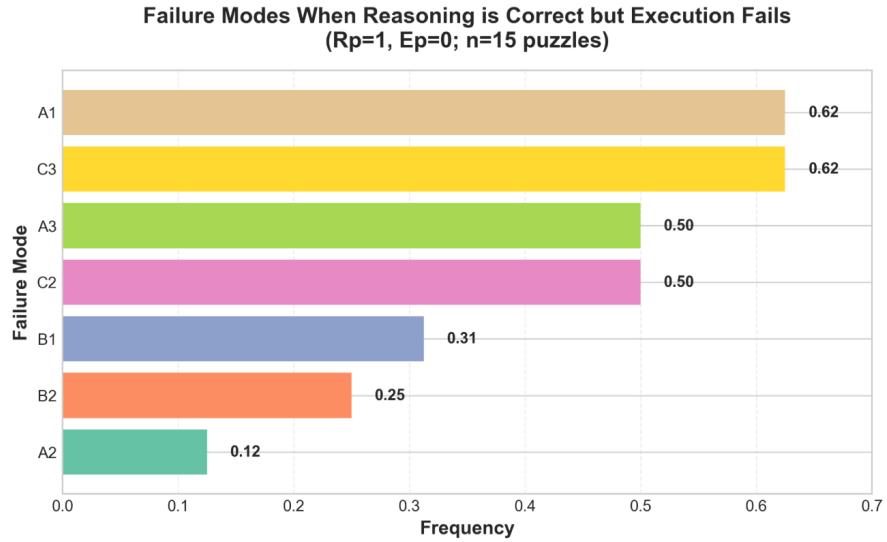


Figure 13. Failure Modes R=1

box/object selection. Some examples get the puzzle right but keep iterating, "Repeated crop/transform steps in the training cases eventually lead to a B-box Misalignment, destroying a previously correct to-be output." These two statements highlight core issues of the pipeline, and of models themselves. Object detection/transform, and misaligned selections. This is due to the difficulty of translating natural language steps to intermediate visuals.

0.5 Strengths

0.5.1 General Steps

While creating visuals to view the intermediate steps can be difficult, the actual steps themselves often represent the "trick" of the puzzle very well. In fact, 45.5% of puzzles were correctly reasoned. This is a stark difference from the 0.9% GPT5-nano with the raw arc data. The annotator repeatedly describes reasoning as "solid" or "generated steps are sound" even when test execution fails. The annotators strictness allowed some general step clarity issues to get past with a correct label, but no steps were labeled as correct reasoning that had 1) a step that would overwrite it's correct progress 2) Any logical errors or 3) Any misrepresentation of the grid, (e.g. saying the red square on the right when its depicted on the left). The annotator only gave correct labels to steps that could plausibly be followed by a human or otherwise to reach correct final output on all examples. This shows that, after comprehensive analysis of all training examples both visually and textually, SoTA models have the capability to understand the reasoning behind ARC puzzles, but may lack the tools to express this reasoning visually or via grid transitions. Here are an example of general steps and it's corresponding puzzle:

Example:

- Step 1: IF the input grid contains exactly two cells whose value is not 0 and those two cells have different color values (call them seed A at (rA,cA) with color colorA and seed B at (rB,cB) with color colorB) THEN identify and record (rA,cA,colorA) and (rB,cB,colorB) as the two single-pixel seeds to drive the pattern.
- Step 2: IF the grid height (number of rows) is greater than the grid width (number of columns) THEN set orientation = horizontal (meaning you will paint full rows).
- Step 3: IF the grid width (number of columns) is greater than the grid height (number of rows) THEN set orientation = vertical (meaning you will paint full columns).
- Step 4: IF the grid width equals the grid height THEN set orientation = horizontal.
- Step 5: IF orientation = horizontal THEN compute period = $rA - rB$; IF period = 0 THEN set period = 1; set start = min(rA,rB); THEN for every integer k such that row = start + k * period lies within the grid, fill every cell in that full row with color = colorA when k is even and color = colorB when k is odd;

leave all other cells (not on those rows) at color 0.

Step 6: IF orientation = vertical THEN compute period = $\text{cA} - \text{cB}$; IF period = 0 THEN set period = 1; set start = $\min(\text{cA}, \text{cB})$; THEN for every integer k such that $\text{column} = \text{start} + k * \text{period}$ lies within the grid, fill every cell in that full column with color = colorA when k is even and color = colorB when k is odd; leave all other cells (not on those columns) at color 0.

Step 7: IF any of the original two seed cells exist in the grid THEN they remain their original color (this is ensured because the filled rows/columns use the seed colors and preserve the seeds' colors).

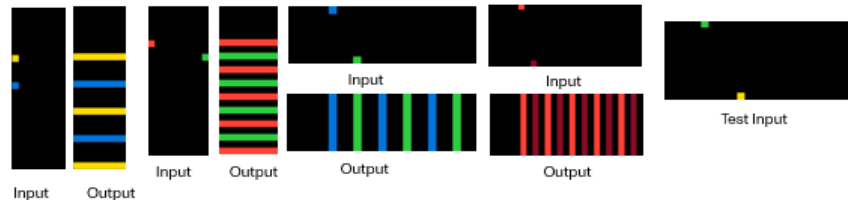


Figure 14. Puzzle 3

These generated steps may be difficult to read for humans, but upon closer inspection, it actually describes the puzzle very precisely, and clearly shows the model understands what changes/doesn't change among all examples. For a generation component with the right tools, (measurement, notepad for computation) this could be a very useful set of steps to achieve full accuracy. There are hundreds of examples of clean, precise, descriptive and accurate steps, but this current generator doesn't currently have the tool set to execute them.

0.5.2 Abstracted Definitions of Objects Lead to Better Generalization

Since the model looks at objects visually, the descriptions of objects/regions/sections become abstracted. In the above example, the model calls the initial input cells "seeds" which infers a starting point. This "seed" description is not tied to any one training input. Having descriptive yet abstracted naming practices for objects allows the generation model to get more information from the step descriptions (seed = starting point for something?) and generalization (the name seed applies to all starting cells in all inputs, as opposed to something like "cell at row 1 column 5.") Other examples of name abstraction in previous examples include "hollow square border," "frame" and "ring" to describe puzzle 2.

0.6 Effective vs Ineffective Training Booklets

This section I will discuss what makes a training booklet that is effective different from one that is ineffective. I define effectiveness of a booklet as such: Given an 100% accurate training booklet (visuals+ step description achieve training output for all booklets) a training booklet is effective if the test prediction is 100% accurate, and ineffective if the test prediction is not accurate. In other words, the booklets "effectiveness" depends of how well it assisted the model in making the correct output. Effective booklets generally have three main qualities. First is that the ordering of steps makes sense to the puzzle. In the second puzzle (Fig 5) we can see that the steps are ordered in a way that makes sense to the "trick" of the puzzle. It measures objects by size, matches certain objects to colors based on this size, and processes them incrementally. Secondly, each visual generated is consistent with it's associated description. This means that when a step says to change a certain portion of the grid, only that portion is changed and the change is faithful to the one described. Thirdly, the model uses the tools it has to make it's reasoning explainable. This is exemplified when the model uses OBSERVE steps correctly. All of these qualities together creates a training booklet with correct visual reasoning. This projects shows that accurate visual reasoning could be a novel expansion on the reasoning ability of current Large Language Models. Now I will give an example of training booklets that lead to an inaccurate test prediction. Below are the general steps, then an example of the generation of one (out of four) of the training booklets, since all of the booklets made the same mistake. In failure categories, ineffective booklets had the most A1, A3, C2, C3 errors, while effective booklets represented the lone (R= 1, E = 1) case. Ineffective booklets also dominate the (R = 1, E = 0) case (reasoning worked, execution didn't) showing that execution (the problems outlined above and in the example) are main causes of failure.

Example 1 Ineffective Booklet Given Inputs and Outputs:

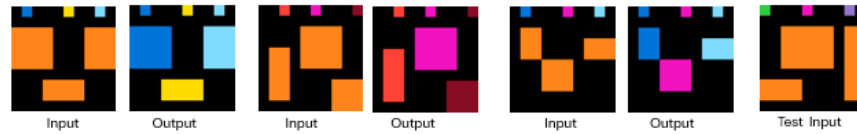


Figure 15. Inputs and Outputs for Puzzle 4

Example 1 Ineffective Booklet, General Steps:

- Step 1: IF a cell is in row 0 AND its color is nonzero THEN do not change that cell's color.
- Step 2: IF a cell's color is not 5 THEN do not change that cell's color.
- Step 3: IF there exists a connected region (connected by orthogonal adjacency) of cells whose color is 5 THEN treat that connected region as one object and compute its horizontal span [min_col, max_col] (the minimum and maximum column indices occupied by that object).
- Step 4: IF there exists one or more nonzero pixels in row 0 at columns C such that min_col ≤ C ≤ max_col THEN select the target column C* equal to the leftmost such column C (the smallest C that satisfies the inequality).
- Step 5: IF a target column C* has been selected for a given connected 5-region THEN set target_color = the color of the pixel at position (row=0, col=C*) (i.e., color at (0,C*)).
- Step 6: IF target_color is defined for that connected 5-region THEN change the color of every cell in that connected 5-region (every cell whose color was 5 and belongs to that object) to target_color.
- Step 7: IF a connected 5-region has no nonzero pixel in row 0 with column in its horizontal span [min_col, max_col] THEN leave that 5-region unchanged.

Steps 1 and 2: OBSERVE

Step 1 and 2 were mislabeled as transform steps but no real transformation was applied. The model cropped to these cells and predicted they wouldn't change, so the grid stayed the same, hence why I omitted the visuals of this step.

Step 3:

IF there exists a connected region (connected by orthogonal adjacency) of cells whose color is 5 THEN treat that connected region as one object and compute its horizontal span [min_col, max_col] (the minimum and maximum column indices occupied by that object).



Figure 16. Step 3 for Puzzle 4

The model did all of the transformations in this step! Since it isn't equipped to draw measurements or highlight the "computed horizontal span" of these rectangular objects, the model "cheats" on this step and goes straight to the output. The generation ends at step 3. Without a meaningful connection between the first row of colors and the correlated rectangular blocks below them, the test model failed to make any transformation. This one example highlights several of the weaknesses of an ineffective booklet.

- The booklet processes too many objects at once. In this example, the LLM finished all of the objects in one step because it matched all objects, since the condition was to compute the horizontal span of all objects.

- The reasoning in the visuals of the booklet is lacking. Since there is limited tool set for depicting measurement, the model failed to visually portray the true reasoning of the step description. This is partially due to the fact that step 3 is also mislabeled as a TRANSFORM step instead of OBSERVE. If it were an OBSERVE step, the model could draw bounding boxes over the computer horizontal span of each object. The LLM that handles test booklet generation misunderstands the "trick" of the puzzle despite seeing all training booklets because the reasoning is not properly conveyed in them.
- The booklet doesn't have the correct number of steps. Since all of the objects have been processed, and the output has been reached, the booklet generation stops. Two versions of this solver I made have two different object processing techniques, and it is unclear which one is more useful. The first version never transforms the same object in multiple steps, and the second stops when all objects have been processed. The main difference between these two versions is that one often produces too many steps, while the other produces too few.

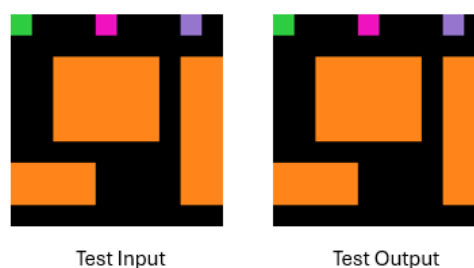


Figure 17. Test prediction using Ineffective Booklet for Puzzle 4

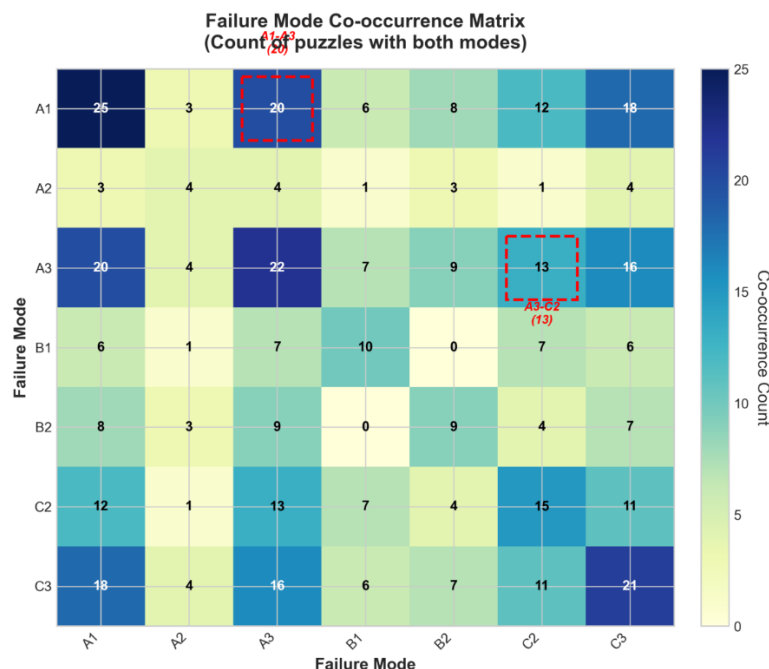


Figure 18. Co-occurrence Matrix of Most Common Errors

0.7 Future Improvements

The co-occurrence map is shown above. The main clusters are circled in red for your convenience. It shows the main issues tend to cluster together, suggesting this is an issue with the implementation itself and not

failures that come up on a case by case basis. This is why it is vital to make a new version with fixes for every major failure model mentioned. Because A1/A3/C2/C3 account for the majority of failures when reasoning is correct, the next version will focus explicitly on these four visual failure modes via fixing the object matching to be more precise. To do this, I need to find a "sweet spot" with how many objects are processed per step, so not all objects are processed at once (skipping other steps. or adding too many steps (also B1, B2 errors)) I will also fix these four failure models by prompting the OBSERVE/TRANSFORM labeling LLM with the definitions of different observation steps. Some observation steps label, measure, or compute sizes of regions, and in the future versions the model will have context of these types of actions. (C3 failures.) Next to fix these four failure modes, I will give the step generation LLM additional toolcalls to measure, compute and label regions using bounding boxes and to take notes. Notes would be appended to for each step, and passed along with booklets to the test model. (C3, C2) In general, I intend to give the model many more tools to work with, so it's more capable of depicting the text step reasoning in a visual way. In the far future, the goal is to produce enough effective booklets with correct reasoning. With a big enough data base of such examples, I could fine tune a reasoning model to make accurate booklets for any ARC puzzle.

0.8 Conclusion

This paper found that models are great at explaining the solutions to ARC puzzles given an in depth visual and textual analysis of all examples. Even models that historically struggle with ARC are capable of writing accurate and precise steps that define general steps for solving a puzzle efficiently. The main component LLMs struggled with was performing adjustments to the grid that were truthful with the generated steps. (C3) Nevertheless, this experiment shows key insights into what models specifically struggle with when it comes to ARC. Perhaps with better tool use models will be able to solve ARC puzzles consistently. While there are no results suggesting the high accuracy of this algorithm, the system of visual reasoning serves as a distinct explainable representation that could be applied in many formats. It allowed me to do an in depth analysis of where models really struggle on ARC. I intend to repeat this experiment with the full knowledge about ARC, visual reasoning, and modularization gained from the trial-and-error of engineering.

REFERENCES

- [1] S. Gu, Y. Shao, K. Huang, *et al.*, "Imagine while reasoning in space: Multimodal visualization-of-thought," *arXiv preprint*, arXiv:2501.07542, Jan. 2025. [Online]. Available: <https://arxiv.org/abs/2501.07542> [web:4][web:5]
- [2] O. Thawakar, D. Dissanayake, K. More, *et al.*, "LlamaV-o1: Rethinking step-by-step visual reasoning in LLMs," *arXiv preprint*, arXiv:2501.06186, Jan. 2025. [Online]. Available: <https://arxiv.org/abs/2501.06186> [web:7][web:13]
- [3] F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard, "ARC-AGI-2: A new challenge for frontier AI reasoning systems," *arXiv preprint*, arXiv:2505.11831, Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2505.11831> [web:8][web:11][web:14]
- [4] J. Berman, "How I got the highest score on ARC-AGI again swapping Python for evolutionary test-time compute," *Jeremy Berman's Substack*, Sep. 16, 2025. [Online]. Available: <https://jeremyberman.substack.com/p/how-i-got-the-highest-score-on-arc-agi-again> [web:22][web:23]
- [5] Z. Carver, "zoecarver/saturn-arc: A visual reasoning approach to ARC-AGI-2," *GitHub repository*, Aug. 9, 2025. [Online]. Available: <https://github.com/zoecarver/saturn-arc> [web:28][web:31]
- [6] [Isabella Shapland], "[ARC-AGI v2: Explainable Reasoning Traces for GPT5]," unpublished manuscript, 2025.
- [7] ARC Prize Foundation, "ARC-AGI: Abstraction and Reasoning Corpus for Artificial General Intelligence," *ARC Prize website*, 2025. [Online]. Available: <https://arcprize.org/arc-agi>
- [8] Y. Feng, Y. Xu, L. Qin, Y. Wang, and W. Che, "Improving language model reasoning with self-motivated learning," *Proc. LREC-COLING*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.07017> [web:3][web:24]